

Trabalho 1 – Aprendizagem Automática

Realizado por:

Tiago Pinto 54718

João Loios 55469

Introdução

O nosso objetivo com este trabalho é implementar os algoritmos KNN e Naive de Bayes para problemas de classificação que permitam a integração com o ambiente scikit-learn. As classes a desenvolver devem permitir criar o modelo, aplicar o modelo e calcular o desempenho.

Pretende-se testar a implementação através da sua aplicação a diversos conjuntos de dados, avaliando o desempenho dos modelos para diferentes hiperparâmetros.

Implementação

Para realizar o nosso programa, dividimos este em 2 ficheiros:

- trabalho.py: ficheiro onde as 2 classes estão implementadas.
- main.py: ficheiro que serve para correr o programa.

trabalho.py

Neste ficheiro estão implementadas ambas as classes KNNeighborsUE e NBayesUE que devem ser o mais compatível possível com o ambiente scikit-learn, por isso, os parâmetros de entrada e saída dos métodos permitem a substituição destes algoritmos por outros implementados no scikit-learn. Assim cada classe apresenta os métodos `__init__`, `fit`, `predict` e `score`, e mais alguns que são necessários para implementar as necessidades requeridas.

KNNeighborsUE

Esta classe implementa um classificador k-vizinhos mais próximos (KNN).

- `__init__(self, k, p)`: Inicializa os parâmetros k (número de vizinhos) e p (ordem da norma).
- `fit(self, X_treino, Y_treino)`: Inicializa os arrays `X_train` e `y_train` e retorna a classe.
- `predict(self, X_test)`: Esta função recebe os valores de `X_test` e calcula para cada X desse conjunto a métrica de Minkowski, para depois através do valor de k, encontrar os índices mais próximos. Para encontrar a classe correspondente, utiliza-se a função `max`, que vê qual a classe que se encontra em maior quantidade correspondente aos atributos mais próximos.
- `distance(self, a_array, b_array)`: calcula a métrica de Minkowski.
- `score(self, predictions, Y_test)`: aplica o modelo a cada exemplo de Y e retorna o valor da exatidão.

NBayesUE

Implementa um classificador Naive Bayes.

- `__init__(self, suave)`: Inicializa o classificador com um parâmetro de suavização (suave).
- `fit(self, X_train, Y_train)`: Inicializa os arrays `X_train` e `y_train` e retorna a classe. Calcula as probabilidades das classes, (devido á existência de valores tão pequenos que o computador igualava a 0, arruinando assim a multiplicação usamos a seguinte igualdade: $\log(P(x_1, x_2, \dots, x_n | C)) = \log(P(x_1 | C)) + \log(P(x_2 | C)) + \dots + \log(P(x_n | C))$, trocando assim a multiplicação pela adição, corrigindo o problema). Calcula as média e variância de cada atributo relativamente a cada classe, e guarda-as numa matriz onde cada linha é uma classe e cada coluna é um atributo.

- `predict(self, X_test)`: Calcula-se a soma das probabilidades (usando uma distribuição Gaussiana modificada com os logaritmos) dos atributos para cada classe e guarda-se no array `attributes_probability` e depois seleciona-se a classe que tiver a probabilidade mais alta e guarda-se no array `predictions`. Por fim retorna-se esse mesmo array.
- `class_probability(self, Xn, mean, variance)`: retorna o valor do cálculo da probabilidade seguindo uma distribuição Gaussiana.
- `score(self, predictions, X_test)`: aplica o modelo a cada exemplo de `Y` e retorna o valor da exatidão.

main.py

Este ficheiro serve para ler o conjunto de dados, executar ambos os algoritmos e mostrar os resultados obtidos.

Desempenhos Obtidos

Apresentamos abaixo os desempenhos que obtivemos com o nosso modelo dos 3 conjuntos de dados distintos fornecidos pela professora: “iris”, “rice”, “wdbc”, e mais um conjunto de dados escolhido por nós: “banana_quality”, que encontramos em <https://www.kaggle.com/datasets/l3llff/banana>.

KNNeighbors

Table 1: Dados de treino de KNNeighbors.

p	k	iris	rice	wdbc	banana_quality
1	1	1	1	1	1
1	5	0,955	0,859	0,657	0,962
1	9	0,929	0,865	0,631	0,961
2	1	1	1	1	1
2	5	0,929	0,845	0,646	0,967
2	9	0,938	0,845	0,554	0,961

Table 2: Dados de teste de KNNNeighbors.

p	k	iris	rice	wdbc	banana_quality
1	1	0,895	0,868	0,811	0,974
1	5	0,895	0,847	0,671	0,966
1	9	0,947	0,846	0,524	0,966
2	1	0,921	0,858	0,783	0,973
2	5	0,947	0,821	0,615	0,963
2	9	0,921	0,830	0,566	0,966

NBayesUE

Table 3: Dados de treino de NBayes.

suave	iris	rice	wdbc	banana_quality
1e-9	0,955	0,780	0,782	0,823
1e-5	0,955	0,781	0,808	0,823

Table 4: Dados de teste de NBayes.

suave	iris	rice	wdbc	banana_quality
1e-9	0,868	0,773	0,776	0,841
1e-5	0,868	0,775	0,741	0,841

Conclusão

Ao realizar este trabalho, tivemos mais dificuldades na implementação do NBayes, por haver probabilidades que eram muito próximas a 0. Apesar de haver algumas exatidões que obtivemos terem sido um pouco mais baixas, pensamos que o trabalho foi bem conseguido. Por fim, apesar de desafiador, gostamos de realizar este trabalho, e pretendemos melhorar os nossos conhecimentos e adquirir mais, futuramente.